Is there a simpler approach?

SpqrBridge slides, alternatives

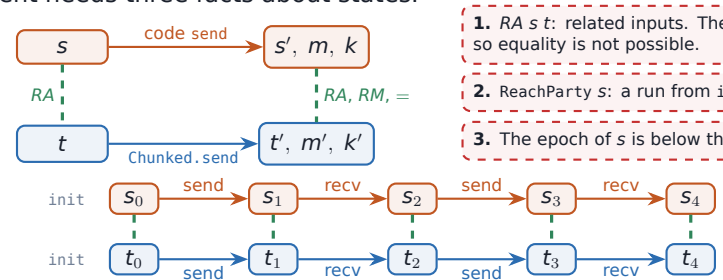
SpqrBridge

November 2026

Can we map `send`, `recv` and `init` function by function, with no state machine?

Can we map `send` to `send`, and stop there?

`send_chunked` is that mapping, from the extracted `send` to `SPQR.Chunked.send`. Its statement needs three facts about states.



1. *RA s t*: related inputs. The types differ, so equality is not possible.

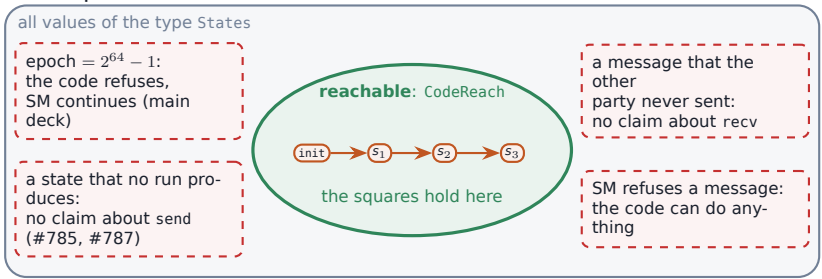
2. `ReachParty` *s*: a run from `init` ends in *s*.

3. The epoch of *s* is below the `u64` limit.

- The output of a send depends on its input state, and that state depends on the earlier calls. So the proof carries *RA* from `init` to step *n*.
- One square for each function, chained along a run, is a simulation between two state machines. The per-function mapping is one step of it.

The mapping holds on reachable states only

For an arbitrary pair of related states, the square of slide 2 is false or not claimed. The premises that repair it are facts about runs.



- `CodeReach` is inductive over `init`, `send` and `recv` of both parties. Without the machine, “reachable” has no meaning.
- The receive premise needs both parties: the message is in the list that the *other* party sent in the same run (`ReachRecv`).
- The bottom-right case is reachable. The game permits it, because SM’s game failed already. An equality of two functions cannot state this.

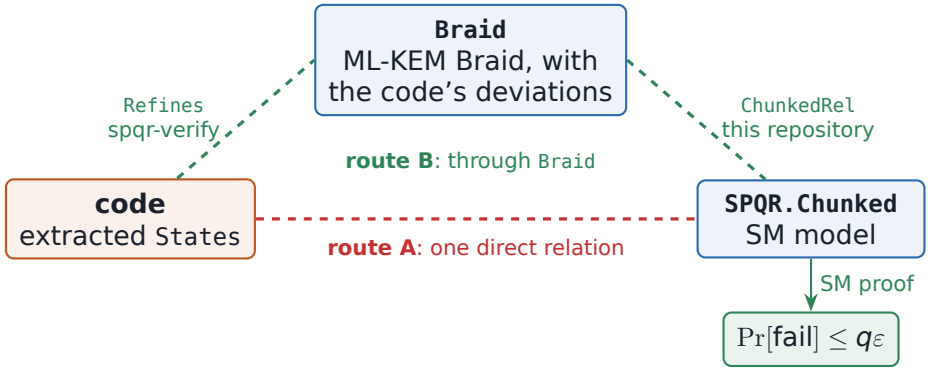
What needs states, and what is a choice

piece	needed?	why
a relation between code states and SM states (*RA*)	yes	the types differ, and `send` takes a state and returns one (slide 2)
a reachability invariant (`CodeReach`)	yes	the step facts fail or are not claimed outside it (slide 3)
a game-level argument (invariant, potential)	yes	the property is about runs of two parties (run deck)
a state map for corruptions (`conc`)	for security	the adversary reads the state (run deck)
`Braid` between the code and SM	a choice	one direct relation is possible (route A, next slides)
the eleven named states	not ours	the Rust type `States` is an enum, and `send` matches on it

- The proposal “map `send` to `send`, `recv` to `recv`, `init` to `init`” is route A. It removes `Braid`. It keeps the first four rows.
- The bridge already contains that mapping: `send_chunked`, `receive_chunked` and the `init` obligation. The transfer theorem takes exactly these three as input.

Two routes from SM’s bound to the code

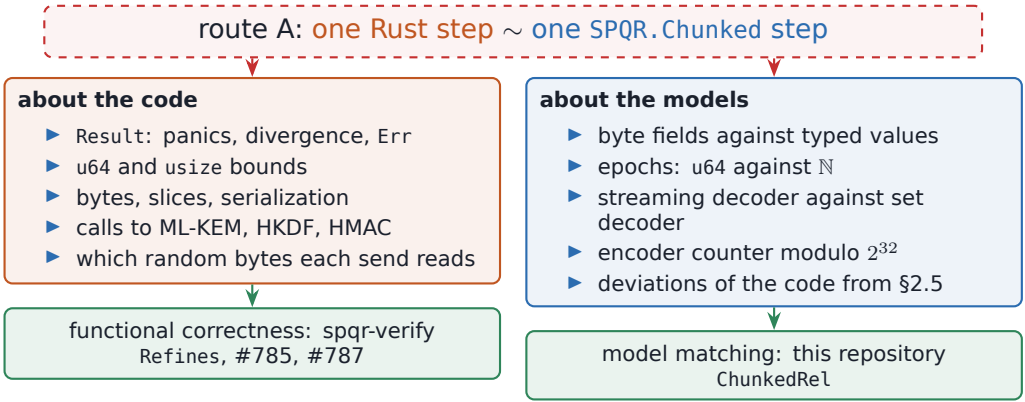
SM proves its bound for the game over `SPQR.Chunked`. To move it, relate each state of the code’s game to a state of SM’s game, step by step.



- Both routes must give one relation *RA* between code states and SM states. Sends are coupled, and receives keep it until SM fails.
- Route B builds it as a composition through `Braid` (main deck).

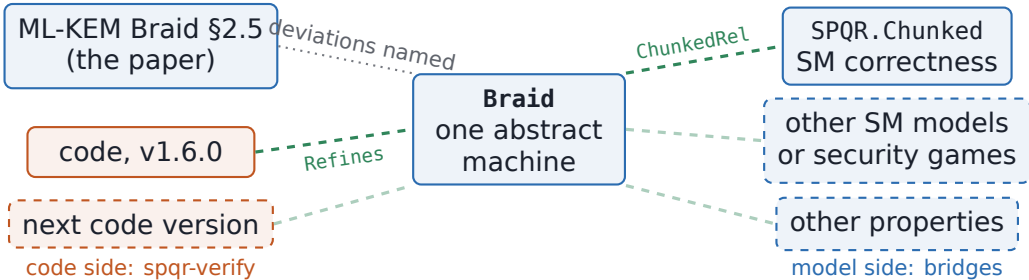
What a direct relation would have to prove

A step of route A relates one Rust step to one SM step. It mixes two kinds of facts.



Route A needs the left column too. It can reuse upstream lemmas, but it states each step against SM's types. Route B states these facts once, against `Braid`. Totality and the randomness fact are still premises there (#785, #787).

Why `Braid` in the middle



- Each repository proves what it owns: facts about the Rust code next to the extraction, facts about SM's model next to SM.
- If `Braid`'s interface stays stable, a change on one side redoes one relation only: a new code version redoes `Refines`, a new model needs a relation from `Braid`.
- Security games are future work. They see leaked randomness, so the coupling must also relate the bytes that each side leaks.
- `Braid` reads like the paper, with each deviation named.
- The cost: one more model to maintain. `ChunkedRel` still needs two code facts: the Rust decoder is lawful, and the encoder counter wraps.